

01 / 18

# Agentic AI

Von der Antwort zur Aktion: KI-Agenten verstehen, bauen und sicher einsetzen

# Was du nach 60 Minuten kannst

- ✓ Chatbot und Agent sicher unterscheiden
- ✓ Die Agentenschleife erklären und selbst skizzieren
- ✓ Werkzeuge, MCP und Agent-zu-Agent-Kommunikation einordnen
- ✓ Entscheiden, wo ein Agent taugt und wo nicht
- ✓ Leitplanken für einen sicheren Einsatz festlegen

# Chatbot oder Agent?

## Chatbot

- Du fragst, er antwortet
- Ein Schritt, dann wartet er
- Wirkt nur im Gespräch
- Du führst das Ergebnis selbst aus

## Agent

- Du gibst ein Ziel, er plant selbst
- Mehrere Schritte in einer Schleife
- Nutzt Werkzeuge: Suche, Dateien, APIs, Code
- Prüft Zwischenergebnisse und macht weiter

# Die Agentenschleife

1. **Ziel verstehen**
   Was genau soll am Ende vorliegen, woran erkenne ich Erfolg?

2. **Planen**
   In kleine, prüfbare Teilschritte zerlegen

3. **Handeln**
   Ein Werkzeug aufrufen: suchen, lesen, rechnen, Code ausführen

4. **Beobachten**
   Ergebnis des Werkzeugs lesen und bewerten

5. **Entscheiden**
   Weitermachen, Plan anpassen oder anhalten und fragen

# Bausteine eines Agenten

- ✓ Modell: das Gehirn, trifft Entscheidungen
- ✓ Anweisungen: Rolle, Ziel, Regeln, Stil
- ✓ Werkzeuge: das, was er tun kann
- ✓ Gedächtnis und Kontext: was er gerade weiß
- ✓ Leitplanken: was er nicht darf oder nur mit Freigabe

# Die Schleife als Code

python

```
def run_agent(ziel, tools, max_schritte=8):
    verlauf = [{"role": "user", "content": ziel}]
    for _ in range(max_schritte):
        antwort = llm(verlauf, tools)          # Modell entscheidet
        if antwort.tool_call is None:
            return antwort.text                # fertig
        ergebnis = tools[antwort.tool_call.name](**antwort.tool_call.args)
        verlauf.append({"role": "tool", "content": ergebnis})
    return "Abbruch: Schrittlimit erreicht"
```

Vereinfacht. llm() steht für den Aufruf eines beliebigen Modells mit Werkzeugunterstützung.

# Agenten-Simulator: die Schleife live

LIVE html · direkt editierbar Zurücksetzen

```
<button id="go">Agent starten</button>
<pre id="log">Noch nichts passiert.</pre>

<script>
  const werkzeuge = {
    rechnen: (a) => a.x * a.y,
    wochentag: () => ['So','Mo','Di','Mi','Do','Fr','Sa'][new
Date().getDay()]
  };

  // Simuliertes Modell: entscheidet pro Schritt, was zu tun ist
  function modell(verlauf) {
    if (verlauf.length === 1) return { tool: 'rechnen', args: {
x: 80, y: 10 } };
    if (verlauf.length === 2) return { tool: 'wochentag', args:
```

Vorschau

Agent starten

Noch nichts passiert.

**Jetzt du:** Ändere das Schrittlimit von 5 auf 2 und starte erneut. Was passiert und warum ist das nützlich?

Das Modell ist hier simuliert, die Schleife ist echt. Mit einem echten Modell ändert sich nur modell().

# Ein Werkzeug beschreiben

json

```
{
  "name": "wetter_abfragen",
  "description": "Liefert das aktuelle Wetter für eine Stadt",
  "input_schema": {
    "type": "object",
    "properties": { "stadt": { "type": "string" } },
    "required": ["stadt"]
  }
}
```

Das Modell wählt Werkzeuge anhand ihrer Beschreibung. Gute Beschreibungen sind der halbe Erfolg.

# MCP: ein Standard für Werkzeuge

Das Model Context Protocol (MCP) legt fest, wie ein Agent Werkzeuge und Datenquellen anspricht. Statt für jedes Programm eine eigene Anbindung zu bauen, spricht der Agent mit jedem MCP-Server auf dieselbe Weise. Das senkt den Aufwand und macht Werkzeuge austauschbar.

# A2A: Agenten sprechen miteinander

Während MCP einen Agenten mit Werkzeugen verbindet, regelt A2A (Agent-to-Agent) die Zusammenarbeit zwischen Agenten, auch über Anbietergrenzen hinweg. So kann ein Recherche-Agent einem Schreib-Agenten Arbeit übergeben, ohne dass beide vom selben Hersteller sein müssen.



# Vier bewährte Muster

- ✓ Einzelagent mit Werkzeugen: der einfache Start
- ✓ Plan und Ausführung trennen: erst planen, dann abarbeiten
- ✓ Orchestrator mit Spezialisten: ein Chef verteilt an Fachagenten
- ✓ Mensch in der Schleife: Freigabe bei kritischen Schritten

# Wann lohnt sich ein Agent?

## Gut geeignet

- Das Ergebnis lässt sich prüfen (Tests, Abgleich)
- Klarer Rahmen und klares Ziel
- Schritte sind rückgängig zu machen
- Wiederkehrende Aufgaben mit festem Muster

## Vorsicht

- Unklares Ziel oder wechselnde Regeln
- Nicht umkehrbare Aktionen: Zahlungen, Löschen, Versand
- Sensible Daten ohne klare Zuständigkeit
- Ergebnisse, die niemand prüfen kann

# Sicherheit: fünf Leitplanken

1. **Minimale Rechte**
   Nur die Werkzeuge und Daten, die die Aufgabe braucht

2. **Freigabe bei Risiko**
   Zahlungen, Löschen, Versand nur nach menschlicher Bestätigung

3. **Limits setzen**
   Schrittzahl, Laufzeit und Kosten begrenzen

4. **Protokollieren**
   Jede Aktion nachvollziehbar speichern

5. **Fremde Inhalte misstrauen**
   Webseiten, Mails und Dokumente können versteckte Anweisungen enthalten

# Modelle wählen: Kriterien statt Namen

- ✓ Teste an deiner eigenen Aufgabe, nicht an fremden Ranglisten
- ✓ Kosten pro erledigter Aufgabe, nicht pro Anfrage
- ✓ Geschwindigkeit: kleine Modelle für einfache Schritte
- ✓ Kontextlänge und Werkzeugzuverlässigkeit
- ✓ Datenschutz, Standort und Vertrag des Anbieters

QUIZ

# Was unterscheidet einen Agenten wesentlich von einem Chatbot?

A Er hat ein größeres Sprachmodell

B Er führt in einer Schleife selbst Schritte mit Werkzeugen aus

C Er antwortet immer schneller

D Er braucht keine Anweisungen

QUIZ

# Welche Aktion sollte ein Agent nur mit menschlicher Freigabe ausführen?

A Eine Webseite lesen

B Eine Zusammenfassung schreiben

C Eine Überweisung auslösen

D Eine Datei umbenennen

# Übung: Dein erster Agent auf dem Papier

1. **Aufgabe wählen**
   Wiederkehrend, prüfbar, mit klarem Ergebnis

2. **Ziel in einem Satz**
   Woran erkennst du, dass es fertig ist?

3. **Werkzeuge festlegen**
   Welche brauchst du, welche bewusst nicht?

4. **Leitplanken notieren**
   Was darf er nie allein tun?

5. **Test planen**
   Fünf Beispiele, bei denen du das richtige Ergebnis kennst

# Von der Theorie zum eigenen Agenten

Im nächsten Schritt bauen wir einen Agenten mit echten Werkzeugen, testen ihn und sichern ihn ab.

Weiter mit Reihe 2: KI-Agenten bauen